

LinPack

A Way to Manage Linux Packages more Efficiently

Bruno Raposo
Fábio Silva

Supervisors: Cátia Vaz, ISEL
Bruno Lourenço

Final report written for Project and Seminary
BSc in Computer Science and Computer Engineering

December 2025

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

LinPack

A Way to Manage Vulnerable Linux Packages more Efficiently

50553 Bruno dos Santos Raposo

50536 Fábio Miguel Guerreiro da Silva

Supervisors:

Cátia Vaz, ISEL

Bruno Lourenço

Final report written for Project and Seminary
BSc in Computer Science and Computer Engineering

December 2025

Abstract

The 2022/2555 (NIS 2) directive from the European Parliament demands that all Member States guarantee a high level of cybersecurity on their information technology infrastructures, promoting the adaptation of technologies that allow an early detection and efficient prevention of cybernetic attacks, and motivating the study of new solutions, especially focused on automatic or semi-automatic tools.

Since the majority of the recent strategies for detecting and monitoring potential cybernetic vulnerabilities rely heavily on manual analysis or checks performed by static tools, a decrease on the efficiency of these processes is observed, resulting in the prolonged exposure of computer systems to threats. With this problem in mind, we propose the LinPack system.

LinPack is a system capable of collecting information about security vulnerabilities and installed Linux packages in a intuitive and easily understandable structure, based on *knowledge graphs*, allowing for a faster identification and effective monitoring of any security flaws present in technology infrastructures built upon this system.

This project is the result of intense study and development of support methods and tools for vulnerability recognition and monitoring of vulnerable packages of a system, including the storage of collected data into a non-relational database which will allow for a clearer organization of the information and an effective management of the contents, due to the possibility of performing more complex queries.

The LinPack system is supported by a web application that enables the visualization of the relations between the packages that are installed in the system and the vulnerabilities that affect them.

Keywords: LinPack; cybersecurity; Linux; *knowledge graphs*; non-relational database.

Resumo

A diretiva 2022/2555 (NIS 2) do Parlamento Europeu exige a garantia da existência de um elevado nível de cibersegurança nas infraestruturas informáticas dos seus Estados-Membros, promovendo a adaptação de tecnologias que permitam a deteção antecipada e a prevenção eficaz de ataques cibernéticos e motivando o estudo de soluções inovadoras, especialmente focadas em ferramentas automáticas ou semi-automáticas.

Sendo que maior parte das estratégias atuais para a deteção e monitorização de potenciais vulnerabilidades cibernéticas dependem fortemente em análises manuais ou verificações realizadas por ferramentas estáticas, observa-se uma diminuição significativa na eficiência destes processos, resultando na exposição prolongada de sistemas informáticos a ameaças. Com este problema em mente, propomos o sistema LinPack.

O LinPack é um sistema capaz de agregar informações sobre vulnerabilidades de segurança e pacotes instalados em sistemas Linux numa estrutura intuitiva e facilmente interpretável, baseada em *knowledge graphs*, permitindo assim uma identificação rápida e uma monitorização eficaz de quaisquer falhas de segurança presentes em infraestruturas tecnológicas desenvolvidas com base neste sistema.

Este projeto é o resultado do estudo e desenvolvimento de métodos e ferramentas de suporte ao reconhecimento de vulnerabilidades documentadas e monitorização de pacotes vulneráveis presentes num sistema, incluindo o armazenamento dos dados obtidos em uma base de dados não relacional, que permitirá uma organização das informações mais clara e uma gestão dos conteúdos mais eficiente devido à possibilidade da realização de consultas mais complexas.

O sistema LinPack é suportado por uma aplicação web que permite a visualização das relações entre os pacotes instalados no sistema e as vulnerabilidades, recentemente divulgadas, que afetam estes mesmos.

Palavras-chave: LinPack; cibersegurança; Linux; *knowledge graphs*; base de dados não relacional.

Contents

List of Figures	vii
List of Tables	ix
List of Algorithms	xi
List of Acronyms	xv
1 Introduction	1
1.1 Outline	2
2 Requirements	3
3 Background	5
3.1 Logs and Vulnerabilities	5
3.1.1 DPKG Logs	6
3.1.2 Vulnerabilities and CVEs	7
3.2 Web Semantic	7
3.2.1 RDF + RDFS	8
3.2.2 Ontologies	9
3.2.3 SPARQL	12
3.2.4 Knowledge Graphs	13
4 LinPack	15
4.1 Overview	15
4.2 System Architecture	16

4.3	LinPack Ontology	17
4.3.1	Vulnerabilities Ontology (CVE)	18
4.3.2	DPKG Logs Ontology	19
4.3.3	Relation between DPKG Logs and CVEs ontologies	25
5	Implementation	27
5.1	Extraction and Parsing	27
5.1.1	Extraction and Parsing of DPKG Log Data	27
5.1.2	Extraction and Parsing of Vulnerability Data	28
5.2	Ontology population	31
5.2.1	Setting up Virtuoso	31
5.2.2	Automatic Insertion of Data into the Graph	32
6	Web Application	39
6.1	Overview	39
6.2	Structure and Data Processing of the Application	39
6.2.1	Obtaining Information	40
6.2.2	Graph Representation	41
7	Final Notes	43
	References	46

List of Figures

3.1	The Semantic Web stack [11]	8
3.2	Example of a RDF triple	9
3.3	Vulnerability Ontology Example	11
4.1	LinPack Architecture	16
4.2	CVE Ontology	19
4.3	Log Event Ontology	22
5.1	Graph Creation	31
5.2	Adding role and permissions to user SPARQL	32

List of Tables

4.1	Detailed comparison between SLOGERT ontology and Log Event ontology	25
-----	---	----

List of Algorithms

5.1	Extract and Parse structured DPKG logs from file	28
5.2	Extraction and parsing of the vulnerability data (NVD API)	29
5.3	Convert and Insert CVE Data as Modular SPARQL Queries	36
5.4	Convert and Insert DPKG Logs as RDF Triples	38

Listings

4.1	SLOGERT representation through classes	20
4.2	SLOGERT representation through data properties	20
4.3	Example of a record of type action	21
4.4	SLOGERT representation of the action record used as example	21
4.5	Example of a record of type action	23
4.6	Log Event representation of the action record used as example	23
4.7	Example of a record of type startup	23
4.8	Log Event representation of the startup record used as example	23
4.9	Example of a record of type conf file	24
4.10	Log Event representation of the conf file record used as example	24
5.1	Insertion of CVE entity with metadata and scoring attributes	34
5.2	Insertion of CVE references and external source metadata	34
5.3	Linking the affected product and its version to the CVE	34
5.4	Definition of the product vendor and ownership relation	35
5.5	Specification of the affected product's version interval	35
5.6	Insertion of Log entity and its corresponding package with metadata and scoring attributes	37

List of Acronyms

<i>IT</i>	Information Technology
<i>NVD</i>	National Vulnerability Database
<i>LLM</i>	Large Language Models
<i>ITU</i>	International Telecommunication Union
<i>DPKG</i>	Debian Package
<i>CVE</i>	Common Vulnerabilities and Exposures
<i>CVSS</i>	Common Vulnerability Scoring System
<i>RDF</i>	Resource Description Framework
<i>FAIR</i>	Findability, Accessibility, Interoperability and Reusability
<i>OWL</i>	Web Ontology Language
<i>W3C</i>	World Wide Web Consortium
<i>KG</i>	Knowledge Graph
<i>TTL</i>	Turtle - Terse RDF Triple Language
<i>SLOGERT</i>	Semantic LOG ExtRaction Template
<i>CEE</i>	Common Event Expression
<i>CPE</i>	Common Platform Enumeration
<i>SVG</i>	Scalable Vector Graphics

Chapter 1

Introduction

In the current landscape, where cyber threats are becoming increasingly sophisticated and frequent, the ability to identify and mitigate system vulnerabilities has become a top priority for organizations in all sectors to ensure the resilience and security of their IT (*Information Technology*) infrastructures.

In this context, Directive (EU) 2022/2555 (NIS 2) [1] further strengthens this imperative by mandating that Member States promote the adoption of cutting-edge technologies to significantly enhance cyberattack detection and prevention. This directive drives innovation within national cybersecurity strategies, emphasizing research and development initiatives aimed at integrating advanced technological solutions, with a particular focus on automated and semi-automated tools.

Given that this is a recent directive, most companies are not yet fully equipped to achieve the proposed objectives. Currently, vulnerability identification and monitoring processes are largely manual or rely on static tools, leaving IT systems highly exposed to cyber threats and jeopardizing the integrity and resilience of corporate infrastructures.

To streamline companies compliance with this directive, this work proposes a solution that structures collected information in a clear, intuitive, and easily interpretable manner, enabling the rapid identification of IT assets vulnerable to specific cyber threats. Our approach, designed for organizational environments, focuses on aggregating data from diverse log datasets, with an emphasis on detecting installed Linux packages and their respective versions. These data points will be cross-referenced with official vulnerability repositories, such as the *National Vulnerability Database* (NVD) [2], to determine whether the identified packages are running vulnerable versions. The validated information will then be integrated into an ontology [3] that models IT assets and vulnerabilities, transforming it into a dynamic knowledge graph [4] that systematically represents their interrelations. Once the knowledge graph is built and stored, it will enable advanced queries and semantic analyzes, allowing for a more efficient system

management.

It is important to emphasize that the focus of this work is not on identifying new, previously unknown vulnerabilities, but rather on monitoring and detecting vulnerabilities that are already known, previously identified, and thoroughly documented. In this context, the vulnerability assessment will be carried out using sources such as the NVD, which generally includes vulnerabilities whose N-day has already passed for a significant amount of time.

In this way, our approach not only streamlines the rapid identification of vulnerabilities but also supports informed decision making, playing a contributing role in the ongoing enhancement of organizational cybersecurity.

Finally, the LinPack system is currently available in the following github repository: <https://github.com/FebioSilva/LinPack-Security-Manager.git>

1.1 Outline

The remainder of this document consists of 6 chapters that collectively provide an understanding of the LinPack system:

- Chapter 2 - Describes the goals and requirements of this project.
- Chapter 3 - Underscores the critical role of logs and documented vulnerabilities within the context of cybersecurity, while also offering a comprehensive overview of the key concepts necessary for understanding the developed project.
- Chapter 4 - Provides the overall information about the system that was created, as well as it's architecture.
- Chapter 5 - Presents the detailed information about the implementation of the LinPack system.
- Chapter 6 - Explores the web application that was developed to provide a simple but complete visualization of the information, giving details about it's implementation and the technologies that were used for it.
- Chapter 7 - Gives some final notes about the project and underscores the main challenges that the team had with it.

Chapter 2

Requirements

In the context of our project, we aim to develop a solution that streamlines the identification and management of vulnerabilities in IT (Information Technology) assets. Our proposed solution leverages the extraction and processing of existing vulnerability data and Linux package logs, transforming them into a structured semantic format to enhance analysis and decision making.

The following obligatory requirements were defined for this project:

1. Extraction and Parsing of information on Linux package vulnerabilities:
 - Develop a program to extract vulnerability data related to Linux packages from official repositories, such as the **National Vulnerability Database (NVD)**.
 - Implement a **parsing** process to transform the received raw data from the **NVD API** into a consistent and structured format, for easy interpretation and processing.
 - Ensure that data includes relevant information, such as vulnerability identifiers (CVE IDs), descriptions, severity levels and affected package versions.
2. Extraction and Parsing of IT asset logs:
 - Develop a **parser** to process log datasets from servers or other IT assets, focusing on the identification of installed Linux packages and their respective versions.
 - The parser must be able to handle the Linux **dpkg** log format and extract relevant information such as:
 - Package names.

- Package versions.

3. Transformation of the data into RDF Triples:

- Develop a transformation process that converts information, such as found vulnerabilities and extracted logs, into **RDF** (Resource Description Framework) **triples**.
- The RDF triples should follow a predefined ontological model, representing entities such as:
 - **IT Assets** (for example, logs and installed packages).
 - **Vulnerabilities** (for example, CVEs, severity levels and affected versions).
 - **Relations** between these two entities (for example, “Package X is affected by vulnerability CVE-2025-1234”).

4. Storage of the generated RDF triples into a knowledge graph database, like **OpenLink Virtuoso**, to allow efficient querying using **SPARQL**.

Optionally, **Large Language Models (LLMs)** can be used to automate and improve tasks such as:

- **Prioritization** of vulnerabilities based on factors such as severity, impact, and the organizational context;
- **Matching** of names between packages mentioned on the IT log system and vulnerable products affected by CVEs.

The use of LLMs aims to enhance efficiency and accuracy in recognizing and managing vulnerabilities, reducing the reliance on manual management and improving the reliability and relevance of the established relationships.

Chapter 3

Background

This chapter emphasizes the importance of the resources used in the system within the context of cybersecurity, namely IT logs in DPKG format and vulnerabilities recorded in official sources. Furthermore, it provides an overview of the semantic technologies relevant to this project, particularly RDF triples and knowledge graphs, which serve as the foundation for structured data representation and analysis. Ontologies are also addressed, since they not only serve as a basis for data representation but also enable the sharing and reuse of semantic information across different systems with distinct purposes.

3.1 Logs and Vulnerabilities

In the field of cybersecurity, threat identification and system protection heavily rely on data analysis. In this context, system logs and vulnerability information play a fundamental role in that analysis.

As per the International Telecommunication Union (ITU), “the log is a repository for records” [5] and an event log record “represents the information stored in the log as a result of receiving notifications or incoming event reports” [6]. They are important because not only they allow the identification of suspicious activities, like an unauthorized access or an uncommon data transfer, but also help to verify the existence of vulnerabilities in the system and ensure that the regulations stipulated by authoritative entities are complied with.

A vulnerability is a weakness in an IT system that can be exploited by an attacker to deliver a successful attack. They can occur through flaws, features or user error, and attackers will look to exploit any of them, often combining one or more, to achieve their end goal [7]. Their role is critical because the identification and correction of a vulnerability is essential to reduce the risk of cyberattacks, like data breaches or

malware infections.

3.1.1 DPKG Logs

The **dpkg** (Debian Package) is an essential tool for installing, removing and managing Debian software packages. It is used in Linux distributions based on this system, like Ubuntu. When a package is installed, removed or updated, the dpkg registers these operations in a log, storing it in the file `/var/log/dpkg.log`.

These logs can be separated in 4 distinct types, each one with their own format and parameters:

- **Action Events:** Events that are registered when an action is performed on a package, such as an installation or an update.
- **State Events:** Events that indicate the current state of a package after an action has been performed on it.
- **Startup Events:** Events that mark the beginning of a dpkg operation, like an installation, removal or upgrading of packages.
- **ConfFile Events:** Events that are stored when a configuration file is detected during an installation or update of a package. The event indicates if the identified file was retained or modified.

The action events have the following format, where the parameter "action" is the action that was performed on the package. It could either be an installation, a removal, an update, a trigger activation, or a configuration:

```
<timestamp> <action> <package name>:<arch> <installed version> <available version>
```

The state events present the following format, where the parameter "state" represents the current state of the package. It could either be installed, not-installed, config-files, half-installed, unpacked or removed:

```
<timestamp> status <state> <package name>:<arch> <installed version>
```

Additionally, the log that capture the execution of a dpkg operation have the following format:

```
<timestamp> startup archives|packages <type command>
```

Finally, the events associated with a decision on either keeping or restoring a configuration file present the following format, where the parameter "decision" consists on the decision that the system made (install or keep):

```
<timestamp> conffile <filename> <decision>
```

3.1.2 Vulnerabilities and CVEs

Given the impact that a vulnerability can have, its identification and categorization are fundamental for the cybersecurity field. To standardize this process, the Common Vulnerabilities and Exposures (CVE) was created, which is a global vulnerability cataloging system. Created and kept by the MITRE Corporation [8], the CVE provides a unique identifier for each public vulnerability, allowing for different security organizations and tools to share a common vocabulary when referring to security flaws. Each CVE entry has a set of standard information that help in understanding and mitigating the vulnerability. The main fields of the CVE include:

- CVE ID: The unique identifier of the vulnerability (ex.: CVE-2025-1234).
- Description: A brief summary of the security flaw, including its effects and possible exploitation methods.
- Affected products: List of vulnerable systems and applications, as well as their affected versions.
- CVSS Score (Common Vulnerability Scoring System): Numeric value that measures the severity of a vulnerability based on factors like its exploitation complexity, impact and authentication requirements.
- External References: Link to external sources like technical reports, advisories or discussions about the vulnerability.

The detailed informations about each individual CVE can be accessed through the NVD [2], which complements the CVE with additional technical data, risk metrics and possible mitigation actions.

3.2 Web Semantic

The Web Semantic is an extension of the Web, proposed by Tim Berners Lee [9], whose objective is to turn the accessible information on the Internet understandable not only

for humans, but also for machines. This approach makes data representation more structured and connected, following the FAIR principles (Findability, Accessibility, Interoperability, Reusability) [10], allowing for autonomous systems to interpret, relate and infer knowledge from distinct sources, as well as share and reuse obtained data.

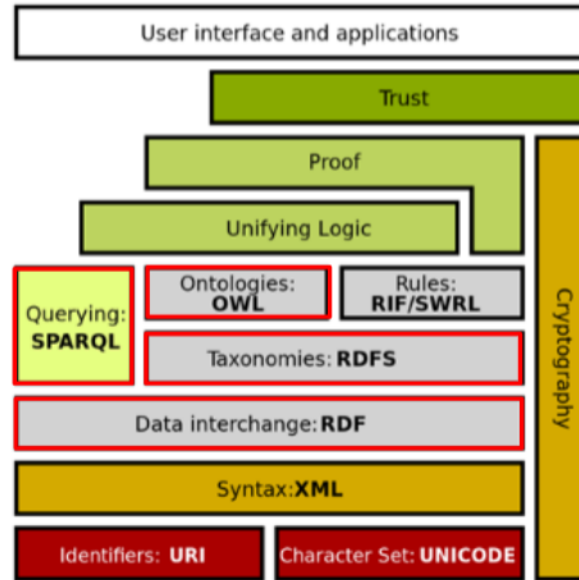


Figure 3.1: The Semantic Web stack [11]

The Web Semantic stack, represented in Figure 3.1, describes the different levels of abstraction and technology that allow web data to be understandable and usable by machines.

In the context of this project, four fundamental layers stand out.

3.2.1 RDF + RDFS

The **RDF (Resource Description Framework)** data model is a data structure used to represent information on the web [12] in a way that is understandable both for humans and machines. The concept of RDF is based on the idea to make assertions about resources (that can be anything) in the form of triples.

A **RDF Triple** is a sequence of three entities that encodes a statement about semantic data in the form of **subject-predicate-object** expressions.

- **Subject:** The resource about which the claim is made. Its the first argument position.
- **Predicate:** The subject's property or feature that is being described. Its the connection between the subject and the object.

- **Object:** The value of the property or the resource to which the subject is related to.

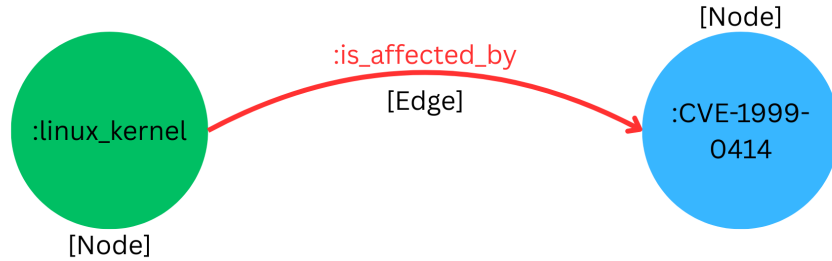


Figure 3.2: Example of a RDF triple

In Figure 3.2 we can see a concrete example of a RDF triple. The subject is represented by `:linux_kernel`, which is the resource about which the claim is made. The predicates, `:is_affected_by`, describes the property that relates the subject to the object. Finally, the object is `:CVE-1999-0414`, which represents the id of the vulnerability that affects the subject. This triple can be read like: "Product `linux_kernel` is affected by a vulnerability identified with the id `CVE-1999-0414`".

Besides RDF, there is also the **RDFS (RDF Schema)**, which is an extension of the RDF that allows the definition of more complex structures to organize and infer knowledge from data [12]. While the RDF focuses on representing simple assertions through triples, the RDFS introduces the capability of defining resource classes, properties and hierarchical relations between them. For example, with RDFS, we could define that `:linux_kernel` belongs to a class of products. This ability to structure vocabularies and relations allows for a more efficient organization of the data and makes the inference of new knowledge much easier, making RDFS an essential tool for the construction of more robust and interoperable semantic systems.

In summary, the RDF and RDFS are fundamental for the construction of the Semantic Web, allowing a more rich and precise interconnection and interpretation of the data.

3.2.2 Ontologies

The term **Ontology** is frequently used in the compute engineering field with regards to knowledge representation. An ontology is a formal description of a group of concepts belonging to the same domain [13]. They are used with the purpose to organize and structure data, in a hierarchical and taxonomic way, allowing an easier interpretation and relationship of information, mainly for a machine.

These structure are mainly made up of:

- **Classes:** Also known as concepts, facts or entities, an ontology class represents a category or set of objects that share common characteristics inside the same domain. A class can have subclasses that represent more specific concepts than the superclass itself.
- **Individuals:** An individual, or instance, of an ontology portrays instances of a class with specified values for the defined properties and established relations with other individuals. Its with these individuals that a knowledge base of the domain of interest is created.
- **Properties:** Also known as roles or slots, the properties define characteristics, attributes and relations of the concepts (classes and individuals) inside a domain. These properties can be data properties, where an individual is associated with an attribute, can be object properties, where two individuals from the same or different classes are associated, or can be annotative properties.

Its worth noting that all the assumptions inside an ontology are represented explicitly, which allows the ontology to be interrogated with automatic reasoning tools to obtain more information and enrich the knowledge base. It also verifies the consistency of data and its properties.

Another important aspect of the ontologies is that they create a common vocabulary and a knowledge that can be shared, integrated and reused. In the cybersecurity field, in particular, many sources produce data and information using different standards and languages. A deeper data integration could allow changing the approach that most systems have towards an attack [14].

How to build an ontology

The first step towards building an ontology is to define a domain, which can be easily done through competency questions, which consist in questions that the ontology should be able to answer [15]. After defining the domain, it is important to search if there are any ontologies created for it, because it is more advantageous reusing and expanding existing resources than to start from scratch. Using as example the management of products affected by vulnerabilities, some examples of competency questions would be:

What is the description of a recorded vulnerability?

Which products are affected by a vulnerability?

What is the severity score given to a vulnerability by the CVSS ?

Which versions of a product are affected by a vulnerability?

Who is the vendor of the affected product?

Having answered all these proposed questions, it becomes possible to define the classes, with the terms that describe the concepts used on the answers, and its properties. Some examples of classes of this ontology are: CVE, Product and Vendor, that represent a vulnerability, an affected product and a vendor of a product, respectively.

The Figure 3.3 presents a drawing of the created ontology with these classes and respective data and object properties.

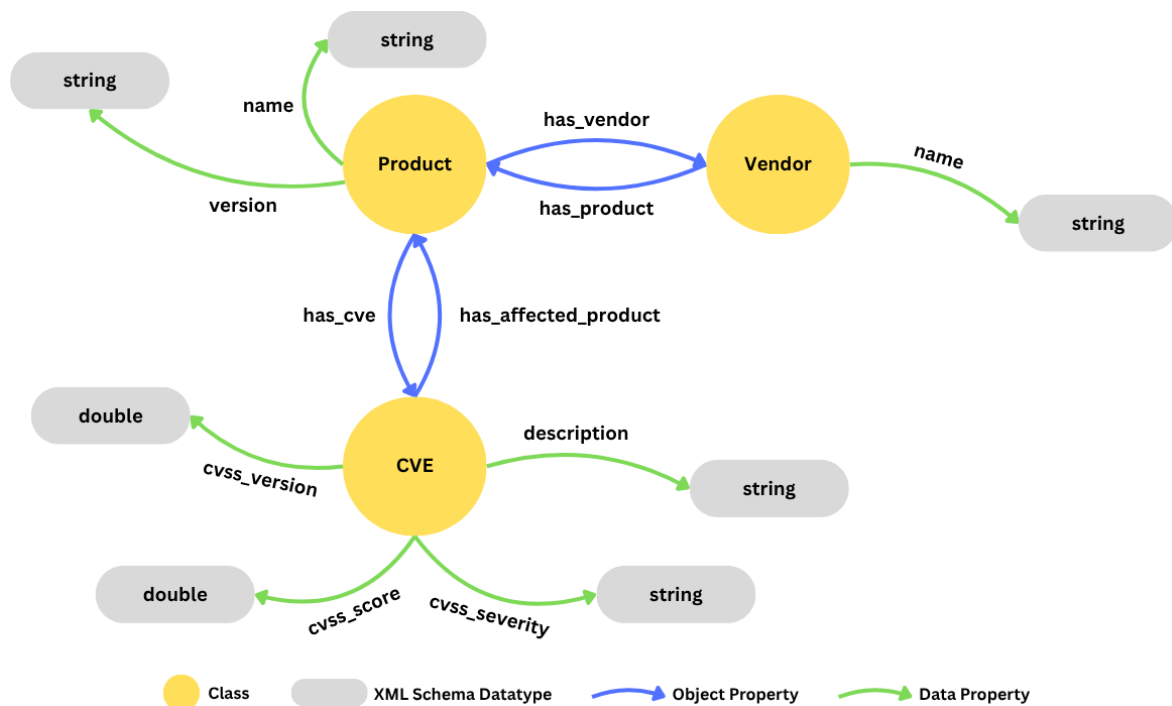


Figure 3.3: Vulnerability Ontology Example

In addition to the definition of the properties, its essential to define their characteristics. The most common are:

- **Cardinality:** Defines how many values a property can have.
- **Type:** Defines the type of the property value (string, int, float, ...).
- **Domain and Range:** Defines the classes that are the subject of a property (Domain) and the classes or values that are its objects (Range).

OWL (Web Ontology Language)

The **OWL** (Web Ontology Language) is a Semantic Web language designed to represent rich and complex knowledge about things, groups of things, and relations between things. OWL is a computational logic-based language such that knowledge expressed in OWL can be exploited by computer programs, e.g., to verify the consistency of that knowledge or to make implicit knowledge explicit [16]. It is used to create **ontologies**.

The main difference between this W3C standard and RDF is that the former allows the definition of cardinalities between classes, enabling, for example, to indicate that a product is affected by at least one vulnerability. Another relevant distinction between these two standard lies in the imposition of restrictions to guarantee the logical consistency of the structure, since OWL establishes these restrictions, while RDF does not.

In addition, the OWL can be used with a **reasoner** which is basically an inference machine that, infers logical consequences from a given set of axioms (RDFS/OWL data). Studies show that OWL reasoners can have a wide range of features. For example, checking the consistency of an ontology, or checking whether a given set of rules applies [17]. It's worth noting that the reasoners complexity will depend, essentially, on the expressiveness of the defined domain, that is, on the number of established relations, concepts and restrictions, and the greater the expressiveness, the greater the complexity of the reasoner.

3.2.3 SPARQL

SPARQL (SPARQL Protocol and RDF Query Language) is a standard language recommended by W3C [18] for consultation and manipulation of data represented in the RDF model.

“Trying to use the Semantic Web without SPARQL is like trying to use a relational database without SQL. SPARQL makes it possible to query information from databases and other diverse sources in the wild, across the Web.”

— Tim Berners-Lee

This statement by Tim Berners-Lee, creator of the World Wide Web and mentor of the Semantic Web, highlights the importance of SPARQL as a technology that facilitates interoperability and intelligent access to distributed data. [19]

SPARQL allows to formulate **declarative queries on knowledge graphs**, treating data as a group of relations (or tuples). Each RDF triple can be interpreted as a relational tuple, bringing SPARQL closer to paradigms used in traditional databases.

A SPARQL language is based on a formally defined query algebra, that specifies operations such as join, union, filter, projection and optional matching, allowing the creation of highly expressive queries. This algebra guarantees semantic completeness and result predictability, being crucial to guarantee interoperability between different implementations and query engines.

One of the most relevant aspects of the SPARQL query is its support for federate queries, that allow the interrogation of multiple distributed SPARQL access points (known as SPARQL endpoints) in a unique query. This ability makes the aggregation of knowledge originating from different data sources possible

3.2.4 Knowledge Graphs

Knowledge graphs (KGs) are defined as graphs of data intended to accumulate and convey knowledge of the real world, whose nodes represent entities of interest and whose edges represent potentially different relations between these entities [20]. These representations utilize formal semantics, which allows computers to process them efficiently and unambiguously [21].

In summary, a knowledge graph is a set of real data, based on semantic modeling, stored in a data base for graphs, allowing a more efficient and flexible integration of the information. The following expression assists on the visualization of this concept: **Knowledge Graph == Ontology + Data + Graph Database.**

In the context of this project, the usage of a knowledge graph makes the correlation between active ITs, installed software packages and known vulnerabilities much easier, providing a scalable and adaptable model for security analysis.

How to build a knowledge graph

To build a well structured and complex knowledge graph, first one must define the data that is going to be stored in the graph. This step can be done through an evaluation of the intended knowledge domain. Following the example defined in section 3.2.2, our knowledge domain will be the products affected by existing and already registered vulnerabilities.

With the knowledge domain defined and the data extracted from the respective sources, we move on to choosing a semantic data model, that is an ontology, that will determine

the way how the data will be presented and related. Having the used example in mind, the ontology that would represent in the best way the entities and relations between products and vulnerabilities would be the one defined in section 3.2.2.

With the semantic model defined, it becomes necessary to determine the database that will store the knowledge graph and insert in it the extracted data on the form of triples. The selected database for this project was OpenLink Virtuoso [22].

Chapter 4

LinPack

This chapter presents an overview of the LinPack system as well as a detailed explanation of the architecture behind it and its data model.

4.1 Overview

LinPack is a system for vulnerability identification and monitoring in ITs. It receives a group of logs, in dpkg format, and verifies if the Linux packages mentioned in them are vulnerable to any type of known and registered cyberattack. This verification is made through a consultation to the NVD [2], which is an official repository that has vulnerability registrations and respective affected products, with the majority being already known and resolved. The obtained information is used by the system to build a detailed knowledge graph, based on a previously defined ontology. This graph is stored in a database, allowing for consultations that make an efficient management of vulnerabilities easier.

The knowledge graph created by LinPack is based on two ontologies defined with the sole purpose to structure and represent, in a precise way, data related to security vulnerabilities (CVEs) and dpkg logs. For the vulnerabilities, it was possible to reuse and expand an existing ontology (`cve.ttl` [23]), incorporating new attributes and restrictions to improve the data representation. In the dpkg log case, the available generic ontologies did not meet the specific needs of the domain, like SLOGERT [24] which in section 4.3.2 we delve deeper into why this ontology cannot be reused, making it necessary to create our own ontology capable of modeling with precision the Linux package management events and their relationships.

4.2 System Architecture

Figure 4.1 presents a detailed view of the LinPack system architecture, separated in three big components: data source, processing and exploration.

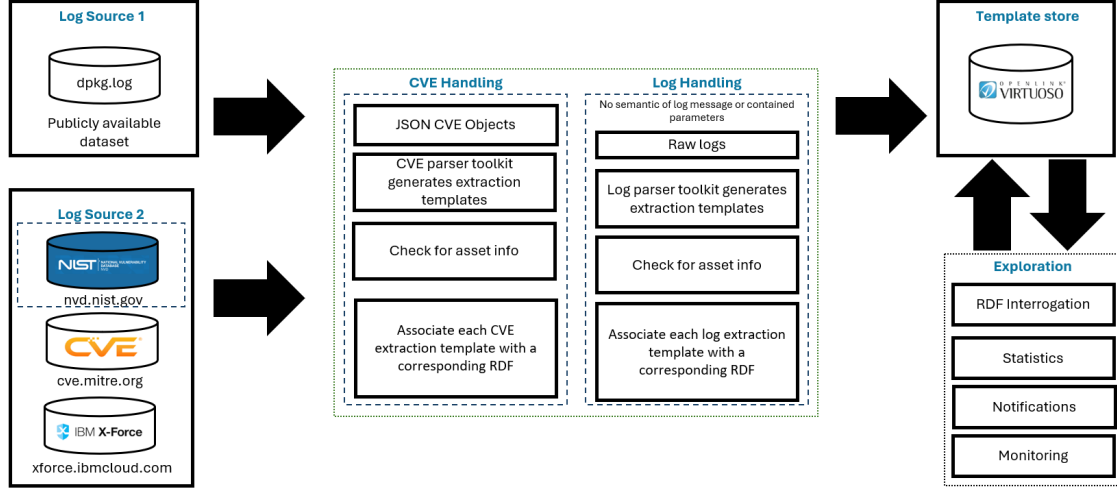


Figure 4.1: LinPack Architecture

Starting from the left, 2 main data sources can be identified:

- **Log Source:** The `dpkg.log` file, which is a public dataset containing records of operations performed on Debian-based Linux system packages that will be used exclusively for the test phase. In the future, this component will be replaced by logs from real ITs.
- **Vulnerabilities Source:** Vulnerability repositories like *National Vulnerability Database* (NVD), `cve.mitre.org` and IBM X-Force. In a first phase of the project, integration is done using only the NVD API, but the architecture predicts future expansion to receive data about vulnerabilities from other sources of information, including European databases that are currently in development.

The central section of the diagram shows how the data processing is done in general, separated by two parallel blocks:

CVE Handling: The vulnerability information is initially extracted from the NVD with the JSON format. This is followed up by the analysis and extraction of relevant entities (e.g., CVE identifier, severity, references, affected products), the validation of the information and, finally, the conversion of each entry to a set of RDF triples according to the CVE ontology.

Log Handling: The raw logs from `dpkg.log` are processed by a parser that extracts events of type `action`, `state`, `startup` or `conffile`. As in the vulnerability block, the extracted data is verified and transformed in RDF triples based on the developed log ontology.

The generated RDF triples are then stored in the **Knowledge Graph**, implemented with the help of *OpenLink Virtuoso* [22], which stores triples and provides a SPARQL endpoint for semantic queries.

Finally on the right, there is the **exploration module**, that allows:

- The execution of **SPARQL queries** to get some insights about the stored data;
- The calculation of statistics of installed packages, vulnerability severity and other indicators;
- The generation of **automatic notifications** about new vulnerabilities affecting an IT's system;
- The **monitoring** of vulnerable packages present on ITs.

This modular and extensible architecture guarantees the existence of a scalable solution for managing vulnerabilities in Linux systems and is aligned with the semantic web principles and the interoperability demands.

4.3 LinPack Ontology

The creation of the LinPack system ontology was an iterative process that involved an analysis of requirements, an evaluation of existing ontologies and a development of new ones when necessary. This process was guided by the necessity to represent, in a precise way, two main domains: the logs of Linux packages (dpkg logs to be more precise) and security vulnerabilities (CVEs), which, when combined, make the domain of our system. Therefore, the semantic representations of both domains make up the LinPack system ontology.

These ontologies were defined with the help of the **Turtle - Terse RDF Triple Language (TTL)** language which is a textual syntax for RDF that allows RDF graphs to be completely written in a compact and natural text form, with abbreviations for common usage patterns and datatypes [25]. We chose this language for its intuitive and readable syntax for humans, in comparison with other alternatives with similar function, like **RDF/XML** [26] and **OWL/XML** [27], while maintaining the compatibility

with RDF structures, something that is not guaranteed by other similar language - **Manchester Syntax** [28].

4.3.1 Vulnerabilities Ontology (CVE)

For the vulnerabilities domain, we chose to reuse and extend an already existing ontology, `cve.ttl`, which provides a robust structure to represent vulnerabilities. This ontology is part of the Unified Cybersecurity Ontology (UCO) [14], developed by the Ebiquity Research Group to support information integration and situational awareness in cybersecurity:

- Main entities like `CVE`, `Product`, `Vendor` and `References`
- Properties like `ref_name`, `ref_source`, and `url`
- Hierarchical relations between vendors, products and vulnerabilities

The main adaptations that were made to this ontology include:

1. **Vocabulary extensions:** Some properties like `pub date`, `base score`, `base severity`, `cvss code`, `cvss version` and `product name` were added. These properties are of extreme importance because they provide critical information about the severity of vulnerabilities and their applicability to specific products. For example, the `base score` and `base severity` allow for a quick evaluation of the potential impact of a vulnerability, while other properties like `product name` are essential to automatically cross vulnerabilities with packages installed on a certain machine - which is fundamental for an efficient risk analysis. Additionally, a new entity, called `Versions`, was incorporated into the ontology which represents the interval of versions of a vulnerable package, that are also vulnerable, and whose properties indicate the limits of the interval and define relations with other entities like `Product` and `CVE`.
2. **Implementation of restrictions:** Semantic restrictions were implemented, such as minimum cardinalities for mandatory fields. These restrictions are essential to guarantee the quality of the data and to prevent critical omissions. For example, making sure that all instances of class `CVE` must have a description and a CVSS score stops important vulnerabilities from being incompletely represented, thus contributing for an automatic and consistent validation of the knowledge graph.

The extension of the existing CVE ontology resulted on the final diagram in Figure 4.2. Instead of modifying the original ontology, we chose to extend it, adding a new class, new properties and domain restrictions necessary to meet the project objectives.

On Figure 4.2, the entities represented with a darker color are from the original ontology, while the ones with a lighter color are the ones that were added in the extension process. This visual distinction makes it more easier to understand the initial state of the ontology and what was the final result after its extension.

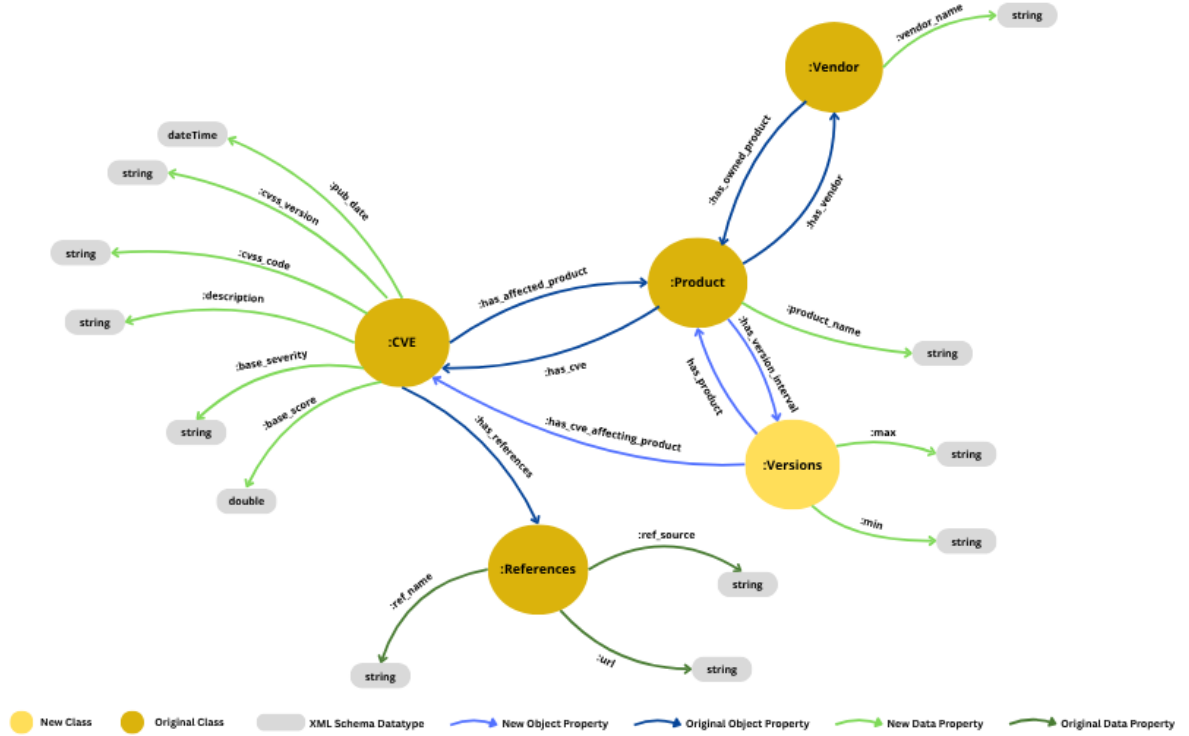


Figure 4.2: CVE Ontology

4.3.2 DPKG Logs Ontology

The creation of the ontology for representing `dpkg` logs followed a different approach than the one used for the vulnerabilities. While on the CVE domain it was possible to adapt an existing ontology, in the case of the `dpkg` logs the possibility of reusing an existing ontology was considered - the **SLOGERT (Semantic LOG ExtRaction Template) Ontology** [24]. However, after deep analysis it was concluded that this ontology did not meet all the specified requirements of this project. The reasons for this decision will be further explored in the following subsection.

SLOGERT Ontology Evaluation

During the survey and analysis phase of ontological solutions to represent log events, the usage of the SLOGERT ontology, that was developed within the scope of the SEPSes project, was considered. This ontology aims to provide a generic and reusable model for representing log events in many domains, based on CEE (Common Event Expression) specification.

The SLOGERT ontology presents a relatively comprehensive structure, containing classes for representing applications, files, users, network addresses and other elements frequently present in log events. However, after a deep analysis we identified some limitations that made us reject the usage of this ontology in the specific context of this project, which is centered in the inconsistency of the representation of the **Parameters** domain.

Ambiguous and inconsistent modeling was observed in the way the parameters are represented. Specifically, the ontology allows for a representation of the same concepts either through *data properties* or *classes*. A practical example of this is the following:

```
:event101 a log:Event ;
          log:hasApp [
            a log:Application ;
            log:app.name "nginx" ;
            log:app.ver "1.18.0"
          ] .
```

Listing 4.1: SLOGERT representation through classes

Another alternative is:

```
:event102 a log:Event ;
          log:parameter.app.name "nginx";
          log:parameter.app.ver "1.18.0" .
```

Listing 4.2: SLOGERT representation through data properties

Both of these representations are semantically equivalent in terms of content, however they both introduce a significant structural ambiguity in the ontological model. The coexistence of multiple paradigms to express the same information - in this case, an application associated with an event log - compromises the semantic consistency of the graph, making automatic inference, ontologic validation and interoperability between instances difficult.

This ambiguous approach raises several practical questions. For example, a SPARQL query that searches applications associated with events will need to have in mind both patterns, which increases query complexity and decreases result predictability. In addition, in environments where logical reasoning is to be applied to entities and their

properties (e.g. infer vulnerabilities based on package versions), the simultaneous use of class instances and literals can lead to incorrect or incomplete inferences

For these reasons, the modeling adopted by SLOGERT isn't adequate for the context of this specific project, where precision and uniformity in the representation of the data are critical factors.

In addition, the ontology also needed to be extended so that all information in each log entry had an adequate logic and semantic meaning.

Let's analyze, for example, the following log record:

```
2025-04-01 11:30:00 install curl:amd64 7.68.0-1ubuntu2 7.68.0-1ubuntu2
```

Listing 4.3: Example of a record of type action

In the generic ontology, this record could be represented in an excessively abstract way, as demonstrated in:

```
:event123 a log:Event ;
    log:hasParameter [ a log:App ;
        log:app.name "curl" ;
        log:app.ver "7.68.0-1ubuntu2" ] .
```

Listing 4.4: SLOGERT representation of the action record used as example

This modeling has three fundamental limitations that compromise its practical utility. First, it omits the specific nature of the registered action (in this case "install") and its timestamp, which are crucial to understand what operation was performed on the system and when it was performed. Secondly, it doesn't capture the package architecture ("amd64") which is essential information in heterogeneous environments with multiple architectures. Finally, and most important, it fails to distinguish the installed version and the available version of the package, which is a critical distinction for the management of vulnerabilities.

These limitations make the generic ontology incomplete for an accurate representation of the dpkg logs in contexts where a detailed analysis of the system state and the performed operations is required. The excessive abstraction results in a loss of relevant semantic information, limiting the ability to perform specific queries and make accurate inferences about the state of packages in the system.

Specific Needs of the DPKG Domain

The requirements for the dpkg logs representations included a detailed typology of events (explicit actions, package states, initialization events and decisions about configuration files), a complete model of packages (with name, architecture and version)

and specific integrity restrictions (mandatory timestamp, cardinalities and accurate data types).

Developed Solution

The custom **Log Event** ontology (`log.event.ontology_v2.ttl`) was developed to meet these necessities, organizing itself around four fundamental types of events, each one with specific properties and well defined relations with involved packages, accommodating the various categories of logs in a differentiated way. This approach mirrors the inherent heterogeneity of dpkg logs, where different operations have distinct characteristics that demand a specific modeling.

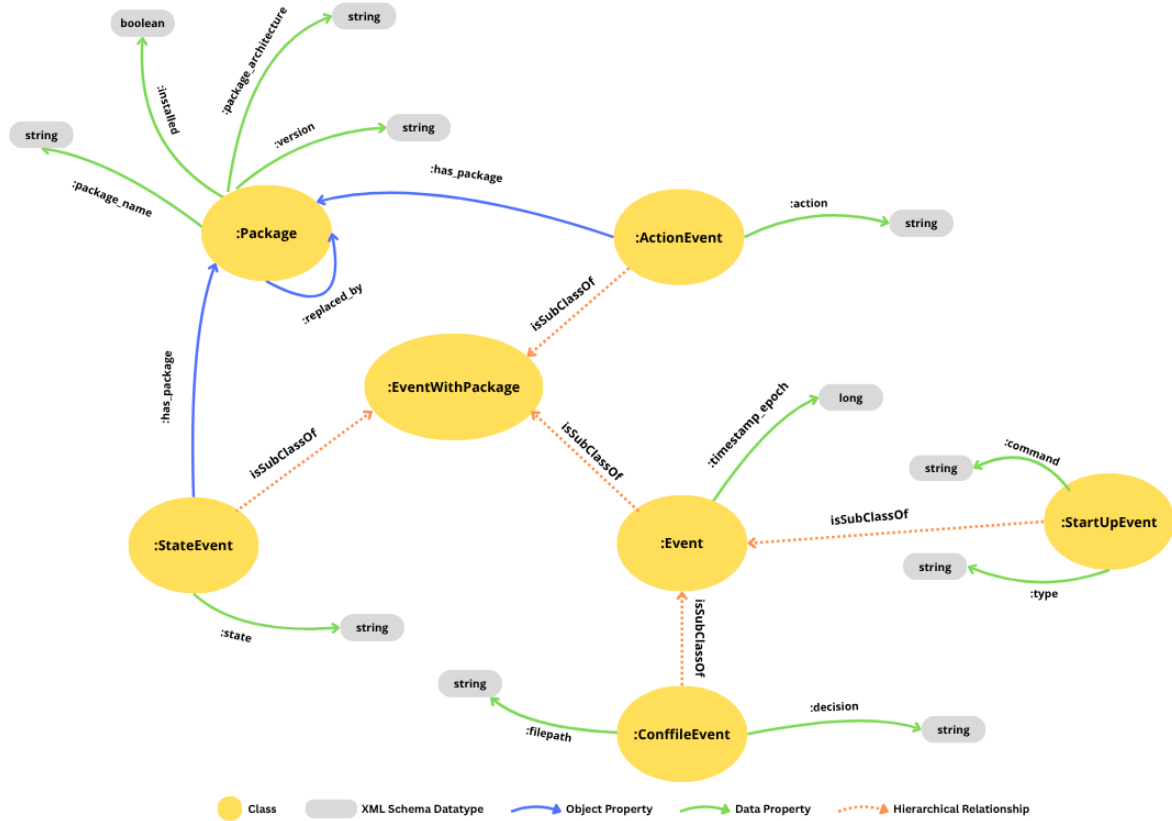


Figure 4.3: Log Event Ontology

For events that involve packages, like installations, deletes or updates (**ActionEvent**) or status changes (**StateEvent**), the property `has_package` establishes a fundamental relation with the **Package** entity. This connection is mandatory in these cases, making sure that all relevant metadata of the package (name, architecture and version) are adequately recorded and associated with the corresponding event. Also, for the **Package** entity, the properties `installed` and `replaced_by` were added to indicate, respectively, if the package is installed in the system and, if upgraded, what updated

package replaced the deprecated one.

However, the ontology recognizes that not all dpkg events necessarily involves packages. The events of type `ConfFileEvent`, that record decisions about configuration files, and the `StartUpEvent`, that document initialization operations over dpkg, are modeled as independent classes without the obligation of the relation `has_package`. This distinction is crucial to maintain fidelity to the original data and avoid creating artificial or redundant relationships.

Returning to the log dpkg example presented previously:

```
2025-04-01 11:30:00 install curl:amd64 7.68.0-1ubuntu2 7.68.0-1ubuntu2
```

Listing 4.5: Example of a record of type action

On the developed ontology, this event is transformed into a rich semantic representation through the following set of RDF triples:

```
:action_event789 a logs:ActionEvent ;
  :timestamp_epoch 1743679800000 ;
  :action "install" ;
  :has_package :curl_7_68_0_1ubuntu2 .

:curl_7_68_0_1ubuntu2 a logs:Package ;
  :package_name "curl" ;
  :package_architecture "amd64" ;
  :version "7.68.0-1ubuntu2" ;
  :installed True .
```

Listing 4.6: Log Event representation of the action record used as example

This modeling has several advantages over the generic approach:

- The specific class `ActionEvent` clearly identifies the type of operation
- All relevant properties are captured in a structured way
- The relationships between events and packages are explicitly modeled

For example, an initialization log:

```
2025-04-01 09:15:00 startup archives unpack
```

Listing 4.7: Example of a record of type startup

is represented in the ontology as:

```
:startup_event1 a :StartUpEvent ;
  :timestamp_epoch 1743498900000 ;
  :operation_type "unpack" ;
  :target "archives" .
```

Listing 4.8: Log Event representation of the startup record used as example

Similarly, a decision log about configuration files:

```
2025-04-01 14:20:00 conffile /etc/nginx/nginx.conf keep
```

Listing 4.9: Example of a record of type conffile

is modeled as:

```
:conffile_event1 a :ConffileEvent ;  
  :timestamp_epoch 1743517200000 ;  
  :filepath "/etc/nginx/nginx.conf" ;  
  :decision "keep" .
```

Listing 4.10: Log Event representation of the conffile record used as example

These differences between event classes with and without package association allows for a more accurate and economically efficient representation of the data. Each event type has only the strictly necessary properties, avoiding the semantic pollution that would result from the indiscriminate application of the `has_package` relation to all records.

The ontology structure, illustrated in Figure 4.3, reflects this discriminated approach through the hierarchy of classes and properties. The built-in OWL constraints ensure the correct application of these principles, validating automatically that:

1. Events that require packages must have the `has_package` relationship
2. Events that do not involve packages cannot have the `has_package` relationship
3. Each event type has only the properties that are relevant to it

This specific event type modeling is a significant advantage over generic approaches, allowing not only a more trustworthy representation of the original logs, but also more efficient queries and more targeted analyses.

Table 4.1 sum up the comparative advantages of the customized approach (*Log Event Ontology*) over the generic ontology (*SLOGERT Ontology*).

The development of this customized ontology demonstrates that, in highly specialized domains, creating a specific solution may be preferable to adapting generic ontologies. The resulting ontology not only addresses the limitations identified in the generic approach, but also provides a solid base to integrate with the vulnerability system, which is essential for the goals of LinPack.

Appearance	SLOGERT Ontology	Log Event Ontology
Purpose	Generic representation of log events for various systems	Specific representation of dpkg logs for Linux package management
Class Structure	• Generic classes (Event, Parameter) • Focus on network/system logs • No specialization for packages	• Specialized classes (ActionEvent, StateEvent) • Hierarchy for dpkg events • Package-centric model
Properties	• 28 properties (3 relevant) • Generic properties (app.ver, cmd) • No package states	• 10 specific properties • Dedicated properties (current_version, state) • Defined action types (install, remove)
Relations	• Generic relations • No event-package link • Basic cardinality	• More specialized relations such as has-package • Precise cardinality • Well-defined domains and scopes
OWL Restrictions	• Basic constraints • Generic data types • Few validations	• Strict constraints • Specific types • Cardinality validations
Extensibility	• Bounded by generic classes	• Easily extensible to new event types

Table 4.1: Detailed comparison between SLOGERT ontology and Log Event ontology

4.3.3 Relation between DPKG Logs and CVEs ontologies

After developing the two main ontologies for representing DPKG Logs and CVEs, the object property `has_related_product` was created in the LinPack ontology domain, with the sole purpose to associate a package that was mentioned in a log event with a product that is vulnerable to one or more CVEs. This relationship, facilitates the identification of vulnerabilities affecting specific packages within an IT system, while also establishing a clear semantic connection between the two ontologies.

Chapter 5

Implementation

This chapter presents an in-depth exposition of the different phases and processes involved in the creation of the LinPack, with the objective of developing an effective system for identifying and monitoring vulnerabilities.

5.1 Extraction and Parsing

To obtain information about dpkg logs and existing vulnerabilities - essential for the development of a complete knowledge graph - it became necessary to develop programs capable of extracting this data from its respective sources and transforming it into a format suitable for its interpretation and efficient use in the system.

5.1.1 Extraction and Parsing of DPKG Log Data

To make the extraction, interpretation and organization of the information in the `dpkg.log` file, algorithm 5.1 was created, in Python.

This algorithm processes a received filepath to the dpkg logs file, generating a list of objects where each object represents a registered event of type **action**, **state**, **startup** or **conffile**. For that, each line of the file is analyzed and compared with predefined patterns corresponding to each log type. Once the type of event present in the line has been identified, the respective relevant properties are extracted, such as the package name, package version, among others, in the case of logs of the **action** type. This information is organized in an object where each type of event is indicated by the property **type**, and is then added to the final list of processed events.

Algorithm 5.1 Extract and Parse structured DPKG logs from file

Data: Log file path

Result: Parsed log entries stored in a structured format

```
1 Initialize: parsed_logs  $\leftarrow$  empty log list, log_id  $\leftarrow$  1
2 Open log file for reading
   foreach line in file do
3   Match line with action_pattern
     Match line with state_pattern
     Match line with conffile_pattern
     Match line with startup_pattern
4   if action_match is found then
5     Extract fields from regex groups
       if action is install then
6       | Append install log to parsed_logs
7     else if action is upgrade then
8       | Append upgrade log with old and new versions
9     else if action is remove or purge then
10      | Append removal log with old version
11     else
12      | Append other action log
13     end
14   else if state_match is found then
15     | Append state change log with package and version
16   else if conffile_match is found then
17     | Append conffile log with filepath and decision
18   else if startup_match is found then
19     | Append startup log with context and command
20   Increment log_id
21 end
```

5.1.2 Extraction and Parsing of Vulnerability Data

To collect, interpret and transform data related to vulnerabilities from the NVD, the following algorithm was created, also in Python:

Algorithm 5.2 Extraction and parsing of the vulnerability data (NVD API)

Data: Initial date *start_date* and final date *end_date*
Result: List of CVE objects programmatically filtered for Linux systems

```

22 Initialize list cve_objects  $\leftarrow []$  Define time window between start_date and end_date
   while Current window  $\subseteq$  total interval do
23   Prepare HTTP request to the NVD API with parameters:
       • pubStartDate, pubEndDate
       • resultsPerPage = 2000
       • startIndex
   while results exist do
       Make HTTP request with API key and parameters Process JSON answer, extract list of vulnerabilities
       foreach Vulnerability in the answer do
           Extract data from the CVE: id, description, severity, references Extract affected CPEs list
           foreach CPE do
               Check if CPE is related to any Linux distributions or Linux kernel
           end
           if at least one CPE is related to Linux then
               Add CVE to the cve_objects list
           end
       end
       Update startIndex Wait 6 seconds (NVD API limitation)
   end
   Proceed to next time window
24 end
25 return cve_objects

```

This algorithm goal is to collect and process all public vulnerability records (*CVEs*) available in the official database *NVD* (*National Vulnerability Database*), within a time range, defined by the user, that must not exceed 120 days (API Limitation).

Due to limitations of the NVD API, namely the fact that the **keywordSearch** parameter only performs a simple textual match on the vulnerability description, and not on structured fields (such as product or vendor name), it was found that this filtering resulted in the omission of multiple relevant *CVEs*. For that reason, we opted for an extraction procedure **without initial filtering by product**, subsequently carrying out local programmatic filtering, based on the analysis of the *CPEs* (*Common Platform Enumerations*) associated with each CVE.

The algorithm starts by iterating, in successive time windows (with maximum duration of 120 days), the defined date interval. For each window, requests are made to the NVD API, with a limit of 2000 results per page. The extraction respects the mandatory minimum interval of 6 seconds between requests, imposed by the API itself.

The answer for each request is a JSON document that contains a set of objects that represent CVEs. For each CVE, the following relevant elements are extracted and processed:

- Unique identifier (CVE Id);
- Description of the vulnerability (preferably in english);
- Severity metrics, with prioritization for the CVSS version 3.1 (if available), including *baseScore*, *baseSeverity* and *vectorString*;
- External references, like technical warnings or safety reports;
- List of affected CPEs, containing structured information about the vendor, product, version and architecture.

After the collection and analysis of the CPEs, an auxiliary function is applied that determines if the CVE is associated with the Linux system (such as Debian, Ubuntu, Red Hat distributions or even the *Linux Kernel* itself). Only the CVEs that affect a component related to Linux are maintained and organized in the form of structured objects.

An alternative approach was also considered, which was based in the realization of individual requests to the NVD for each package identified in the dpkg logs. This strategy would consist in checking, for each log, if there were any vulnerabilities directly associated with the registered package and only keep the CVE if this association was verified. However, this approach was discarded for 2 fundamental limitations:

- The minimum interval between request (6 seconds) would turn this process extremely slow and impractical in real environments with multiple logs;
- The API request with package name filtering, even in this model, would keep to be based by a literal search in the description of the CVE, which was revealed to be an imprecise and insufficient strategy for the objective.

Like this, the adopted strategy - total extraction followed by a local programmatic filtering - revealed to be more efficient, ensuring more exhaustiveness, reliability and control over the collected data. However, with this strategy, the time to process and treat the data also increased considerably.

5.2 Ontology population

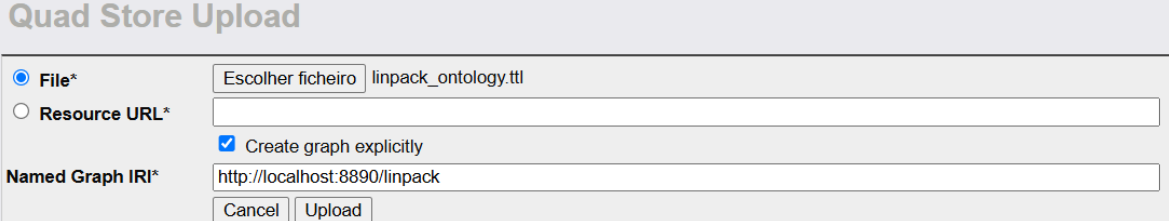
This section describes the process of populating the ontologies developed within the scope of the project, divided into two main parts. The first one addresses how the ontology developed for vulnerabilities (CVEs) is populated and the second one describes the same but for the dpkg log ontology instead. Each subsection details the algorithms and generic methods used to transform the raw data into RDF triples and store them in the Knowledge Graph.

5.2.1 Setting up Virtuoso

Before inserting the data into the Knowledge Graph automatically, it was necessary to prepare the database environment that is going to serve as our local data storage. In order to meet this requirement, we used **OpenLink Virtuoso**, which is a database server that natively supports RDF and provides a complete SPARQL endpoint for read and write operations [22].

For the first step of the set up, the Virtuoso Docker image was downloaded, followed by the creation of a container using that image, that exposes necessary ports (namely 8890 to connect to the web interface).

After the start of the container, the Virtuoso administrative graphical interface was accessed through the endpoint `http://localhost:8890/conductor`, where the ontology that combines the CVE and DPKG logs ontologies was manually loaded, resulting in the creation of a graph, with the URI `http://localhost:8890/linpack`, as illustrated in Figure 5.1, which would serve as the destination for the insertion of the RDF data.



The screenshot shows the 'Quad Store Upload' window in the Virtuoso web interface. It has a light gray header with the title 'Quad Store Upload'. Below the header, there are two radio buttons: 'File*' (selected) and 'Resource URL*'. To the right of the 'File*' button is a text input field containing 'linpack_ontology.ttl'. Below the radio buttons, there is a checkbox labeled 'Create graph explicitly' which is checked. At the bottom, there is a text input field labeled 'Named Graph IRI*' containing the URI 'http://localhost:8890/linpack'. At the very bottom, there are two buttons: 'Cancel' and 'Upload'.

Figure 5.1: Graph Creation

In the next step, it was necessary to properly configure the permissions of the user responsible for inserting data into the graph. By default, the user designated as SPARQL, which will be the user used for insertions, does not have the necessary privileges to execute write operations through SPARQL queries.

To overcome this limitation, the `SPARQL_UPDATE` role was assigned, which allows modifications to be made to the data in a graph of the database created. Additionally, explicit write permissions were granted to the database, ensuring that the user has all the necessary privileges to perform queries for insertion and not just reading. These configurations are described in Figure 5.2.

The screenshot shows the 'Editing Account' interface for user SPARQL. The 'Default Permissions' table is highlighted with a red box, showing full permissions (r, w, x, t) for Owner, Group, and Users. The 'Account Roles' section is also highlighted with a red box, showing 'SPARQL_UPDATE' selected in the 'Selected' list.

Owner	Group	Users	Idx
r	r	r	t
w	w	w	w
x	x	x	x
t	t	t	t

Available	Selected
administrators	SPARQL_SELECT
nogroup	SPARQL_COMPILE
SPARQL_EXTRACT_FROM_COI	SPARQL_UPDATE
SPARQL_LOAD_SERVICE_DATA	
SPARQL_SELECT_FED	

Figure 5.2: Adding role and permissions to user SPARQL

5.2.2 Automatic Insertion of Data into the Graph

Once the Virtuoso environment was configured properly, we proceeded to implement the logic responsible for the automated insertion of data into the Knowledge Graph. This implementation consisted of transforming the raw data from two different sources - vulnerabilities (CVEs) and Linux system package logs in DPKG format (extracted through the parsers developed and explained in previous sections) — into RDF triples compatible with the defined ontology, and their respective insertion into the previously created graph.

Two independent modules were developed, each responsible for the population of its respective ontology. In both cases, the `SPARQLWrapper` library was used, which is a Python interface to communicate with SPARQL endpoints [29]. This library allowed for the dynamic construction of SPARQL insertion queries (`INSERT DATA`) and sending them directly to the Virtuoso endpoint.

- Subsection **CVE Ontology Population** describes the strategy adopted to trans-

form the raw data resulting from the extraction and parsing of CVEs into RDF triples and subsequent insertion into the knowledge graph.

- Subsection **DPKG Log Ontology Population**, similarly to the previous Subsection, explains in detail the implementation made to transform the raw data of DPKG Logs into RDF triples so that they can be inserted into the knowledge graph.

Each subsection describes the processing methods used, the data modeling according to the defined ontological structure, and the algorithms that support the generation and submission of queries SPARQL for populating the graph.

CVE Ontology Population

The population of the ontology related to the vulnerabilities was based on the data already extracted, parsed and processed from the API of National Vulnerability Database (NVD). Population of the ontology consists essentially on mapping that information into the defined ontology model, transforming it into RDF triples and inserting them directly on the Virtuoso's graph.

The RDF triples were generated dynamically from the relevant data of each vulnerability: CVE's identifier, description, severity (CVSS), affected CPEs and external references.

During testing, we identified a limitation of the Virtuoso SPARQL endpoint related to the maximum number of characters accepted in a single query. When the number of RDF triples for a CVE became too big, especially for entries with many references or many affected products and versions, the generated query exceeded this internal limit and resulted in errors.

To address this, the insertion logic was adapted by **splitting each CVE insertion into multiple smaller SPARQL queries**. For example:

- One query inserts the base CVE entity (ID, description, CVSS score, etc.);
- A second query inserts all references and links them to the CVE;
- A third query adds the affected products and their associated vendors;
- A final query inserts the version intervals for each product.

This modular strategy prevents Virtuoso from exceeding the query size limit and ensures successful data insertion.

The following segments present an example of the SPARQL queries generated for a single vulnerability (of type CVE), illustrating how the extracted and mapped data was divided into blocks and transformed into RDF triples for insertion into the graph:

```
PREFIX cve: <http://purl.org/cyber/cve#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
  GRAPH <http://localhost:8890/linpack> {
    cve:CVE-2023-12345 a cve:CVE ;
    cve:description "This is a test CVE description.\nIt has
multiple lines." ;
    cve:base_score 7.5 ;
    cve:base_severity "HIGH" ;
    cve:cvss_version "3.1" ;
    cve:cvss_code "AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H" ;
    cve:pub_date "2023-10-01T12:00:00"^^xsd:dateTime .
  }
}
```

Listing 5.1: Insertion of CVE entity with metadata and scoring attributes

```
PREFIX cve: <http://purl.org/cyber/cve#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
  GRAPH <http://localhost:8890/linpack> {
    cve:CVE-2023-12345 cve:has_references cve:
ref_nvd_https__nvd_nist_gov_vuln_detail_cve_2023_12345 .
    cve:ref_nvd_https__nvd_nist_gov_vuln_detail_cve_2023_12345 a cve:
References ;
    cve:url "https://nvd.nist.gov/vuln/detail/CVE-2023-12345" ;
    cve:ref_source "NVD" ;
    cve:ref_name "NVD_https://nvd.nist.gov/vuln/detail/CVE
-2023-12345" .
  }
}
```

Listing 5.2: Insertion of CVE references and external source metadata

```
PREFIX cve: <http://purl.org/cyber/cve#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
  GRAPH <http://localhost:8890/linpack> {
    cve:prod_example_product cve:has_version_interval cve:
example_product_vers_1_0-2_0 .
    cve:prod_example_product a cve:Product ;
    cve:product_name "example-product" ;
    cve:has_vendor cve:vendor_example_vendor .
    cve:CVE-2023-12345 cve:has_affected_product cve:
prod_example_product .
  }
}
```

Listing 5.3: Linking the affected product and its version to the CVE

```

PREFIX cve: <http://purl.org/cyber/cve#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
  GRAPH <http://localhost:8890/linpack> {
    cve:vendor_example_vendor a cve:Vendor ;
    cve:vendor_name "example_vendor" ;
    cve:has_owned_product cve:prod_example_product .
  }
}

```

Listing 5.4: Definition of the product vendor and ownership relation

```

PREFIX cve: <http://purl.org/cyber/cve#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
  GRAPH <http://localhost:8890/linpack> {
    cve:example_product_vers_1_0-2_0 a cve:Versions ;
    cve:min "1.0" ;
    cve:max "2.0" ;
    cve:has_product cve:prod_example_product ;
    cve:has_cve_affecting_product cve:CVE-2023-12345 .
  }
}

```

Listing 5.5: Specification of the affected product’s version interval

Algorithm 5.3 makes the transformation of data into RDF triples and builds the insertion query.

Algorithm 5.3 Convert and Insert CVE Data as Modular SPARQL Queries

Data: CVE object with metadata, references, and affected products

Result: SPARQL queries to populate the RDF graph in separate blocks

26 Function:

```
Initialize empty list of SPARQL queries
Extract CVE metadata: identifier, description, severity, publication date
Generate SPARQL query for the main CVE block and add to the query list
if CVE has references then
    foreach reference in references list do
        Create a blank node for the reference with source, URL, and name
        Link reference node to the CVE instance
    end
    Group reference triples into a separate SPARQL block and add to the list
end

Initialize sets to track already inserted vendors, products, and versions
Initialize lists for vendor triples, product triples, and version triples
foreach CPE (affected product) in CVE do
    Extract vendor, product, and version intervals if vendor not yet processed then
        Generate vendor triples and add to vendor list
    end
    if product not yet processed then
        Generate product triples and add to product list
    end
    foreach version interval do
        if interval not yet processed then
            Generate version triples and add to version list
        end
        Link product to version interval and CVE
    end
end
foreach triple group in {vendor, product, version} do
    Split group into chunks (e.g., max 50 triples) foreach chunk do
        Generate a separate SPARQL INSERT query and add to query list
    end
end
return List of SPARQL queries for insertion
```

DPKG Log Ontology Population

Similar to the process described for vulnerability data, to insert the events extracted from the *dpkg* logs into the *Knowledge Graph* it was necessary to convert the initial data into RDF triples that were compatible with the ontology that represents the logs.

To meet this purpose, it was implemented a *Python* module whose main role is to receive as input a previously structured log object (resulting from the analysis described in Subsection 5.1.1) and returns a SPARQL query of type `INSERT DATA`, ready to be submitted to the *endpoint* Virtuoso.

The construction of the query is made in a dynamic way, taking into account the type of event recorded (e.g. `action`, `state`, `conffile` or `startup`). Based on this type, the appropriate semantic properties are included, according to the structure defined in the ontology, ensuring the logical consistency of the insertion.

Each event is identified by an unique URI and it's instantiated as a member of the corresponding class (for example `logs:ActionEvent`, `logs:StateEvent`), associating the properties such as `logs:timestamp`, `logs:has_package`, `logs:architecture`, `logs:has_action` among others, depending on the context.

One aspect worth noting is that the association between both ontologies is created while populating this ontology. In the creation of a RDF Triple to represent a package mentioned in a log, the object property `has_related_product` is introduced with a similar name to the package name as a value, because not all products affected by a CVE have the same name of the corresponding package mentioned in a log event. This similarity is obtained through a separate query that retrieves from the database all vulnerable products whose names are contained within the mentioned package name. The most similar product is then determined using the Levenshtein algorithm, which calculates the similarity between two strings based on their Levenshtein distance [30].

The following segment presents a simplified example of a query SPARQL generated for an event of type `action`:

```
INSERT DATA {
  GRAPH <http://localhost:8890/linpack> {
    logs:log1095 rdf:type logs:ActionEvent ;
                  logs:timestamp "2025-06-10T13:12:04"^^xsd:dateTime
    ;
                  logs:action "install" ;
                  logs:has_package logs:aspera_faspex_5_0_11 .

    logs:aspera_faspex_5_0_11 rdf:type logs:Package ;
                              logs:package_name "aspera_faspex" ;
                              logs:package_architecture "amd64" ;
                              logs:version "5.0.11" ;
                              logs:installed True ;
                              linpack:has_related_product cve:prod_aspera_faspex
    .
  }
}
```

Listing 5.6: Insertion of Log entity and its corresponding package with metadata and scoring attributes

The transformation of data and its insertion on the graph is shown in Algorithm 5.4.

Algorithm 5.4 Convert and Insert DPKG Logs as RDF Triples

Data: Log entry `log_obj`, matched product `matched_product`

Result: SPARQL INSERT query string

```
27 Define SPARQL prefix block sparql_prefix
    Create log identifier log_id from log_obj["log_id"]
    Set log_type as capitalized type with suffix "Event"

28 Initialize SPARQL string with header, timestamp, and log type

29 if log type is action or state then
30 |   Generate package_uri using sanitized package name and version
    |   Add corresponding action/state line and package URI to SPARQL
31 else if log type is conf file then
32 |   Add filepath and decision fields to SPARQL
33 else if log type is startup then
34 |   Add context and command fields to SPARQL
35 Remove trailing semicolon and close the log triple block

36 if log contains package info then
37 |   if action is install or trigproc then
38 |   |   Add package definition as installed = True
    |   |   Add related product if available
39 |   else if action is remove or purge then
40 |   |   Add package definition as installed = False
    |   |   Add related product if available
41 |   else if action is upgrade then
42 |   |   Define old_package_uri using replaced version
    |   |   Add old package info with installed = False & replaced_by
    |   |   Add new package info with installed = True
43 Close the SPARQL graph and return the query string
```

Chapter 6

Web Application

In this chapter we will talk about the web application created to represent the stored data in the Virtuoso database, in order to provide a simple and complete view of the relevant information for efficient management of vulnerabilities in installed packages.

6.1 Overview

This web application was developed with the purpose to complete the knowledge graph construction and to provide a more accessible and intuitive interface for users of the LinPack system. This application allows for a visual and interactive exploration of the relations between Linux packages installed in the system of one of the user's ITs and known vulnerabilities that affect them, with the goal to turn their management easier. This representation is based in data previously inserted in the knowledge graph and stored on the Virtuoso endpoint.

The application connects to the endpoint provided by Virtuoso that allows SPARQL queries to be made to the database and obtains, through predefined queries, the relevant data for the representation of installed Linux packages, existing vulnerabilities that affect them and the relations between both of these entities. Finally, the application builds a graph visualization with the information consulted, allowing the application of filters to demonstrate more specific information.

6.2 Structure and Data Processing of the Application

The logic of the application can be divided in two main phases:

6.2.1 Obtaining Information

To obtain the necessary data for the representation of the installed packages and vulnerabilities that affect them, the application will interact with the Virtuoso through the SPARQL endpoint, to which will make HTTP request with the queries that it wants to execute. The data, returned by the endpoint, will be in the JSON format and will later be used in the representation of the information. The queries used in our application were:

- **logsAndCVEs**: Collects all logs that mention installed packages in the system, as well as all CVEs that affect them. This query can also receive, as a parameter, a year to filter the search, obtaining only the CVEs, that affect installed packages, documented in the indicated year. This query is used to represent the graph with the installed packages, the vulnerabilities that affect them and the relations between these entities;
- **highestSeverityCVEsQuery**: Obtains the 5 most severe documented CVEs that affect one of the packages installed in the system. This query is used in the statistics window of the application;
- **countCVEsPerProductQuery**: Gets the number of CVEs that affect each installed and vulnerable product. Again, this query is used in the statistics of the system window.

During data collection, the need to filter installed packages that were within a certain range of vulnerable versions, defined by each CVE, arose. Although we initially considered applying this filter directly in the SPARQL query, we quickly observed that this approach was not viable. This is due to the fact that the Virtuoso engine that executes SPARQL queries to the database only supports SQL-style functions within the query itself, which greatly limits the capability of making complex comparisons between versions. This limitation is particularly relevant on the dpkg log packages context, whose versions can have different formats (e.g. `1.2.3`, `1.2.3-4ubuntu1`, `1:1.2.3~beta`), demanding a specialized comparison logic which is not natively supported by Virtuoso, that only allows us to make literal comparisons between strings.

Given these limitations, we chose to apply the version comparison logic directly in the Web application layer. To guarantee a rigorous comparison, the `dpkg-compare-versions` library [31] was used, ensuring total compatibility with the different possible formats. The adopted strategy consisted in obtaining all the packages associated with a certain vulnerable product and, later, apply locally the version interval verification, discarding

the packages whose version was not included between the `min` and `max` values defined for the vulnerability.

6.2.2 Graph Representation

After the reception of the information in a structured format, its transformation to an internal representation composed of *nodes* and *links*, which will compose the visual graph, is carried out.

Each entity of the system - such as packages, versions, CVEs and products - is transformed in an individual node. The semantic relations between these entities, previously defined in the ontology, are converted to connections that reference both nodes.

During this process it is ensured that each node has useful properties for visualization, such as labels, colors and categories. Additionally, the data is normalized to avoid repetitions and to maintain graph coherence.

The graph is built using the D3.js library [32], which is a widely used tool for dynamic visualization of data on the browser. This library is based on SVG (Scalable Vector Graphics) [33] and interactive events. This technology allowed to represent, in a clear way, the graph entities (such as packages, versions, vulnerabilities and logs) through nodes and their relations through edges. The resulting visualization is dynamic, allowing the user to interact with the graph elements, zoom in or zoom out, move nodes freely and obtain more information about each entity when hovering the cursor over a node.

Since the application is based on the Javascript language, we chose to use Webpack [34] to organize and compile the code. This tool allows to manage dependencies, group modules and optimize loading the application in the browser. This choice allowed the code to be organized into modules, as well as the possibility of integrating new libraries or frameworks in the future. Furthermore, we used Webpack to serve as a proxy, enabling the web application to connect with the database without having problems related to CORS (Cross-Origin Resource Sharing).

For the structural and visual representation of the application we used HTML5 and CSS3.

Chapter 7

Final Notes

In the development of this project, it was necessary to deepen knowledge in areas outside our usual domain, namely in the Semantic Web, RDF triples, ontologies, knowledge graphs and in the SPARQL query language. Besides that, we explored many technologies that contributed significantly for the realization of the proposed system, such as the OpenLink Virtuoso database for the semantic storage of the information, as well as the libraries SPARQLWrapper and D3.js, used, respectively, for the interaction with the database and for the visual representation of data on the web application. This study allowed us to acquire a solid knowledge about new concepts and tools, expanding our technical skills and providing a valid practical experience on the application of semantic technologies in real environments.

After concluding the development of the basic and main features of the LinPack system, we considered interesting using, in the future, a **LLM (Large Language Model)** to **prioritize vulnerabilities** so that, in an automatic way, the LLM will warn the user, based on the severity of the vulnerability and the number of products vulnerable to it, which extracted vulnerabilities should be treated first and **improve association of log packages with affected products**, so it improves the matching between packages mentioned in logs and products vulnerable to an extracted CVE through the search of specific similarities in both entities, allowing for a more precise association since this matching is currently made resorting to a search by products whose name is contained in the package name, which does not provide totally precise results.

The creation of an ontology that would be adequate to the goals of our system proved to be a challenging task due to the specification and format of the data obtained from the respective sources and due to the complexity in the relationship between defined individuals. One of the main obstacles on this phase was to find an efficient way to associate both ontologies that make the LinPack ontology - the CVEs ontology and the DPKG logs ontology. Many approaches to this problem were explored, such

as the creation of a new entity on the LinPack ontology domain that would connect simultaneously with affected products and mentioned packages, or even a relationship between packages present on logs and the versions of the vulnerable products. After many iterations, the adopted solution consisted on the one mentioned in previous chapters - establish a direct connection between the registered package on the log and the product affected by the CVE.

The decision to represent versions as intervals revealed to be equally challenging. This choice was made in order to align the modeling with the way data is made available in the NVD, which frequently indicates, along with the affected product, the vulnerable versions through a minimal version and a maximum version. However, there are cases in which no minimum or maximum versions are defined, using only the character '*' or, sometimes, '-' as an indicator of the version. To deal with these situations, we chose to assume that, in these cases, all versions of the mentioned product are affected by the CVE. This approach, although it may include some redundant information, was preferable to omit potential relevant vulnerabilities. The improvement of this aspect of the system, with the definition of more consistent intervals in these specific cases, will be made in the future.

The filtering of versions of packages installed in the system, although it is currently only performed after executing the query in the developed web application, it should, ideally, occur before or during this execution. Such approach would allow the system to work in a more autonomous and complete way, without depending on the visualization layer to ensure the consistency of the results. However, due to the complexity associated with the correspondence between the package mentioned in the logs and a vulnerable version, specially in cases where there is an individual insertion of an event, this feature will be considered future work for the project.

Finally, due to the confidential nature of the type of data used in this project, the system is not used nor it was tested in real scenarios, with the automatic insertion of new logs and CVEs into the database. In order to simulate one of these real scenarios, we implemented a DPKG log generator (`dpkgGenerator`) that, at predefined time intervals, generates synthetic logs and stores them in a file named `dpkg_synthetic.log`. In parallel, a DPKG log receiver (`dpkgReceiver`) was implemented to periodically read the contents of this file, extract the relevant information and insert it into the database. When the web application is in use, it periodically sends requests to the database to check for newly added logs. If new information is detected, the application displays a warning to the user indicating that a page refresh is required to view the updated data. Although this approach works for receiving new logs, the same cannot be said for the CVEs, as associating them with existing logs is significantly more complex. Due to

this complexity, we have decided to leave this aspect of the system as future work.

References

- [1] Centro Nacional de Cibersegurança (CNCS). *Diretiva SRI 2 (NIS 2)*. Available: <https://www.cncs.gov.pt/pt/diretiva-sri-2-nis-2>. Last Accessed: 2025-03-03. Centro Nacional de Cibersegurança (CNCS).
- [2] National Institute of Standards and Technology (NIST). *National Vulnerability Database (NVD)*. Available: <https://nvd.nist.gov>. Last Accessed: 2025-03-03.
- [3] Dieter Fensel and Dieter Fensel. *Ontologies*. Springer, 2001.
- [4] Leslie F Sikos. “Cybersecurity knowledge graphs”. In: *Knowledge and Information Systems* 65.9 (2023), pp. 3511–3531.
- [5] International Telecommunication Union. “Information Technology—Open Systems Interconnection—System Management: Alarm Reporting Function”. In: (1992).
- [6] International Telecommunication Union. “Information Technology Open Systems Interconnection Structure of Management Information: Definition of Management Information”. In: (1992).
- [7] National Cyber Security Centre (NCSC). *Vulnerability Management*. Available: <https://www.ncsc.gov.uk/collection/vulnerability-management/understanding-vulnerabilities>. Last Accessed: 2025-03-26.
- [8] MITRE Corporation. *MITRE Corporation*. Available: <https://www.mitre.org/>. Last Accessed: 2025-05-29.
- [9] Wikipédia, a enciclopédia livre. *Web semântica*. https://pt.wikipedia.org/wiki/Web_semntica. Last Accessed: 2025-03-19. 2024.
- [10] Martin Boeckhout, Gerhard A Zielhuis, and Annelien L Bredenoord. “The FAIR guiding principles for data stewardship: fair enough?” In: *European journal of human genetics* 26.7 (2018), pp. 931–936.
- [11] Han Li and Tianzhen Hong. “A semantic ontology for representing and quantifying energy flexibility of buildings”. In: *Advances in Applied Energy* 8 (2022), p. 100113.
- [12] Frank Manola, Eric Miller, Brian McBride, et al. “RDF primer”. In: *W3C recommendation* 10.1-107 (2004), p. 6.
- [13] Natalya F Noy, Deborah L McGuinness, et al. *Ontology development 101: A guide to creating your first ontology*. 2001.
- [14] Zareen Syed, Ankur Padia, M Lisa Mathews, Tim Finin, Anupam Joshi, et al. “UCO: A unified cybersecurity ontology”. In: *Proceedings of the AAAI Workshop on Artificial Intelligence for Cyber Security*. 2016, pp. 195–202.

- [15] Glaice Kelly Q Monfardini, Jordana S Salamon, and Monalessa P Barcellos. “Use of competency questions in ontology engineering: a survey”. In: *International Conference on Conceptual Modeling*. Springer. 2023, pp. 45–64.
- [16] W3C. *Web Ontology Language (OWL)*. Available: <https://www.w3.org/OWL/>. Last Accessed: 2025-03-27.
- [17] Konrad Abicht. “OWL Reasoners still useable in 2023”. In: *arXiv preprint arXiv:2309.06888* (2023).
- [18] W3C. *SPARQL 1.1 Query Language*. <https://www.w3.org/TR/sparql11-query/>. Last Accessed: 2025-03-20. 2013.
- [19] Ontotext. *What is SPARQL?* <https://www.ontotext.com/knowledgehub/fundamentals/what-is-sparql/>. Last Accessed: 2025-03-20. 2023.
- [20] Aidan Hogan et al. “Knowledge graphs”. In: *ACM Computing Surveys (Csur)* 54.4 (2021), pp. 1–37.
- [21] Ciyuan Peng, Feng Xia, Mehdi Naseriparsa, and Francesco Osborne. “Knowledge graphs: Opportunities and challenges”. In: *Artificial Intelligence Review* 56.11 (2023), pp. 13071–13102.
- [22] OpenLink Software. *What is Virtuoso? — Virtuoso Documentation*. <https://docs.openlinksw.com/overview/whatisvirtuoso>. Last Accessed: 2025-04-19.
- [23] Unified Cybersecurity Ontology. *CVE Ontology*. Available: <https://github.com/Ebiquity/Unified-Cybersecurity-Ontology/blob/master/cve/cve.ttl>. Last Accessed: 2025-04-29.
- [24] W3C. *The SLOGERT*. Available: <https://sepses.ifs.tuwien.ac.at/ns/log/>. Last Accessed: 2025-04-29.
- [25] W3C. *Turtle - Terse RDF Triple Language*. Available: <https://www.w3.org/TeamSubmission/turtle/>. Last Accessed: 2025-05-03.
- [26] W3C. *RDF/XML*. Available: <https://www.w3.org/TR/rdf-syntax-grammar/>. Last Accessed: 2025-05-03.
- [27] W3C. *OWL/XML*. Available: <https://www.w3.org/TR/owl2-xml-serialization/>. Last Accessed: 2025-05-03.
- [28] W3C. *Manchester Syntax*. Available: <https://www.w3.org/TR/owl2-manchester-syntax/>. Last Accessed: 2025-05-03.
- [29] W3C. *SPARQL Endpoint interface to Python*. <https://sparqlwrapper.readthedocs.io/en/latest/>. Last Accessed: 2025-07-08.
- [30] Wikipedia. *Levenshtein Distance*. Available: https://en.wikipedia.org/wiki/Levenshtein_distance. Last Accessed: 2025-06-26.
- [31] Philipp Matthias Hahn. *dpkg -compare-versions*. Available: <https://pmhahn.github.io/dpkg-compare-versions/>. Last Accessed: 2025-07-08.
- [32] Elijah Meeks and Anne-Marie Dufour. *D3.js in Action*. Simon and Schuster, 2024.
- [33] Jon Ferraiolo, Fujisawa Jun, and Dean Jackson. *Scalable vector graphics (SVG) 1.0 specification*. iuniverse Bloomington, 2000.

- [34] Mohamed Bouzid. *Webpack for Beginners: Your Step-by-Step Guide to Learning Webpack 4*. Apress, 2020.